

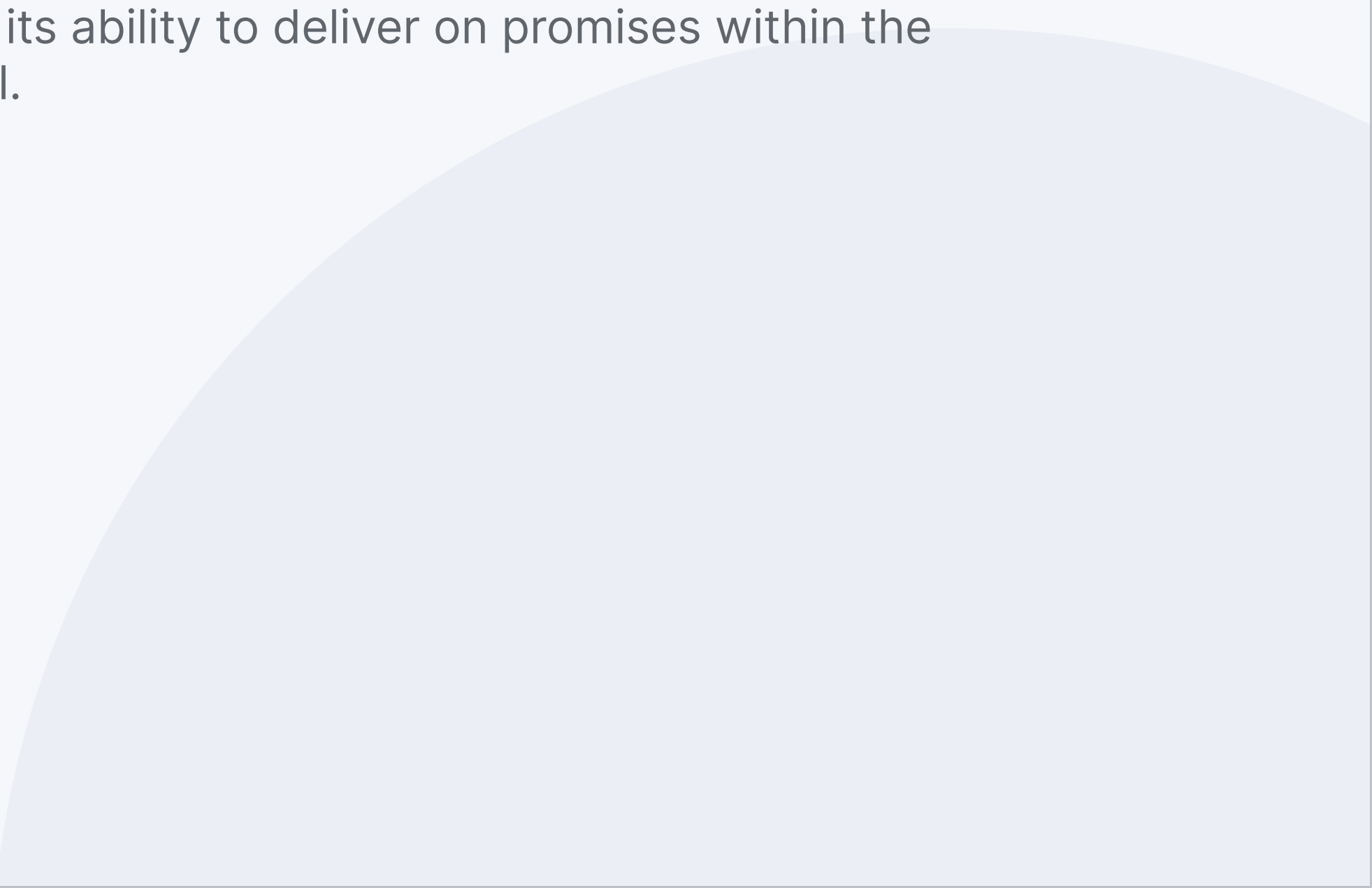


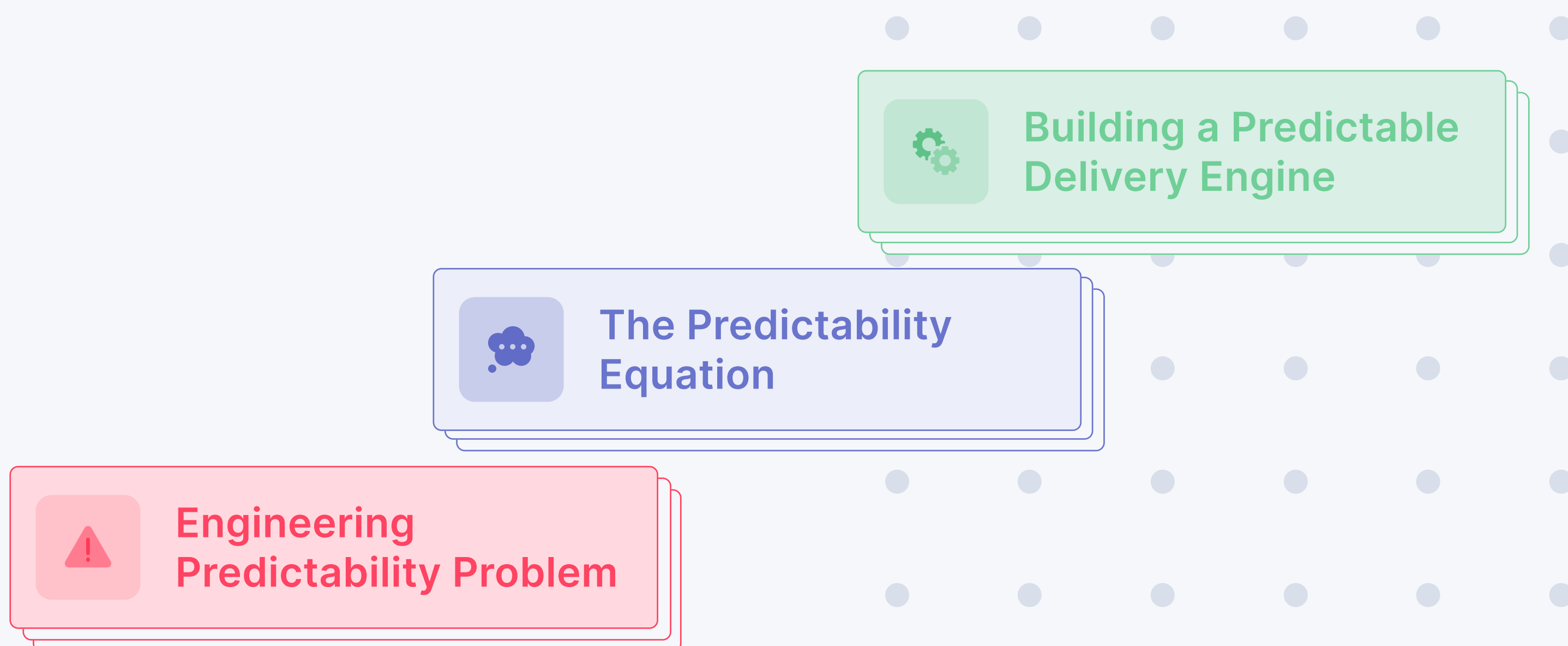
Engineering Leader's Guide to Predictable Project Delivery

Predictable project delivery is the ultimate engineering outcome. The ability to know what you'll deliver, when, is the result of key improvement initiatives and best practices coming together. Its value for engineering orgs of all shapes and sizes—as well as the rest of the business—cannot be overstated.

The role of an engineering leader revolves around the dual mandate of delivering operational excellence and aligning engineering investments to company goals.

These foundational missions underpin a single pursuit: predictable and consistent project execution and delivery. Because engineering is the primary value driver for the organization, its ability to deliver on promises within the agreed-upon timeframe is critical.





In this guide—part five of the five-part engineering success model—you’ll learn all about how to bring all the lessons from the previous Engineering Leadership Guides together and build a predictable project delivery engine within your organization.

What's covered in the pages ahead:

- **Engineering’s Predictability Problem:** The reasons why consistency and stability in project delivery are such elusive outcomes for many engineering organizations.
- **The Predictability Equation:** Diving into the calculus that drives steady project delivery and examining the variables that impact predictability.
- **Building a Predictable Delivery Engine:** Now that you understand the "math," you'll need the right tools to help you solve the equation.

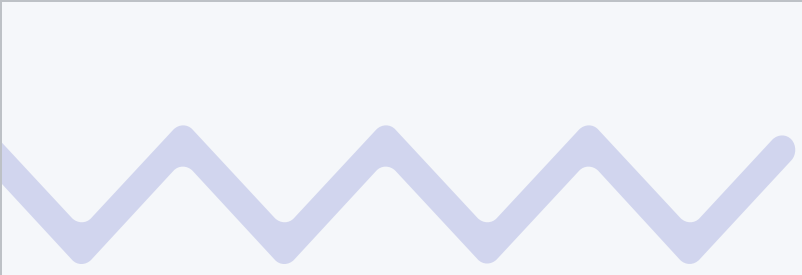


Table of Contents

1. Engineering Has a Predictability Problem

The rest of the organization cannot do their jobs
It's not only the "other teams" that suffer

3. The Predictability Equation

Stability
Velocity
Risk

8. How Syngenta Became More Predictable At Scale

14. Building a Predictable Project Delivery Engine

Visibility, Benchmarks, and Diagnosis
Allocating Resources Correctly and Investing Wisely
Setting Data-Backed Improvement Goals
Adopting Programmable Workflows

18. Conclusion

Engineering Has a Predictability Problem

Universal truths are rare but they do happen:

1. **2+2 = 4**
2. **Han shot first** (He's an outlaw with no compunction about doing what needed doing)
3. **Earth is a globe**

Here's one more universal truth: **no one likes losing face**. Losing face in front of business leaders and executive stakeholders even less so.

Yet, according to LinearB data, **the industry average for planning accuracy is <50%**. Said another way, when engineering leaders scope work and convey delivery timelines to their business counterparts, they routinely miss the mark. Even more troubling is the engineering leader's audience when communicating these *delayed* project timelines, *incorrect* projected delivery dates, and *lack of* progress against stated goals: business leaders and executive stakeholders.

This situation is untenable because engineering is the enterprise's biggest value driver. Too often, this lack of consistency and predictability in project delivery drives undesirable outcomes.

The rest of the organization cannot function

Here are some examples of how a lack of engineering can impact other departments:



A **marketing team** whose job is to drive awareness and demand by promoting new products, features, and enhancements can't plan campaigns effectively, schedule events about new offerings, or publish content that references new functionality if they can't count on delivery dates.



Sales can't effectively or confidently sell a roadmap item or feature vision if delivery consistently slips (they can try, but the customer experience, relationship, and brand reputation will suffer).



Customer success is constantly in limbo as they attempt to convince customers that the vision and feature set sales sold them are "coming very soon" when they're just guessing (read: hoping) that the release comes sooner rather than later.

Engineering Has a Predictability Problem

It's not only the "other teams" that suffer

A lack of predictability also negatively impacts R&D organizations, specifically developer experience (DevEx), a crucial consideration for overall engineering health and productivity. In fact, according to the 2023 **Accelerate State of DevOps Report**, a healthy culture "has a positive impact on all key outcomes." Consistency and good culture are synonymous.

Teams with generative cultures have **30% higher organizational performance** than teams without

Aspect of Culture	Effect on team performance	Effect on organizational performance	Effect on software delivery performance	Effect on operational performance
Westrum's organizational culture	↑↑ Substantial increase	↑↑ Substantial increase	↑↑ Substantial increase	↑↑ Substantial increase
Organizational stability	↑ Minor increase	↑↑ Substantial increase	↓ Minor decrease	⊖ No effect
Job security	↑ Minor increase	⊖ No effect	↑ Minor increase	↑ Minor increase
Flexibility	↑ Minor increase	↑ Minor increase	↑↑ Substantial increase	↑ Minor increase
Knowledge sharing	↑ Minor increase	↓ Minor decrease	↑↑ Substantial increase	↑↑ Substantial increase
User-centrism	↑↑ Substantial increase	↑↑ Substantial increase	↑ Minor increase	↑↑ Substantial increase
Work distribution	↑↑ Substantial increase	↑↑ Substantial increase	↓↓ Substantial decrease	⊖ No effect

Data from 2023 Accelerate State of DevOps Report

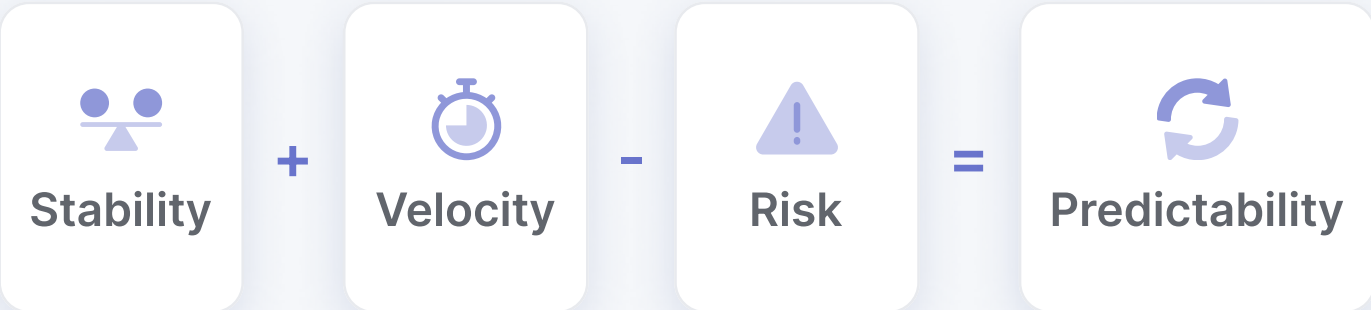
For example, let's take a developer on a predictable team that regularly delivers 85-115% of what they plan to build and can accommodate reasonable fluctuations and additions in their workload--a fairly standard practice on any given engineering team.

Generally speaking, that developer knows what any given day will look like with very few surprises (great DevEx). They can set reasonable, achievable objectives and even confidently set stretch goals without creating unsustainable workloads and burnout culture associated with constant deadline stress.



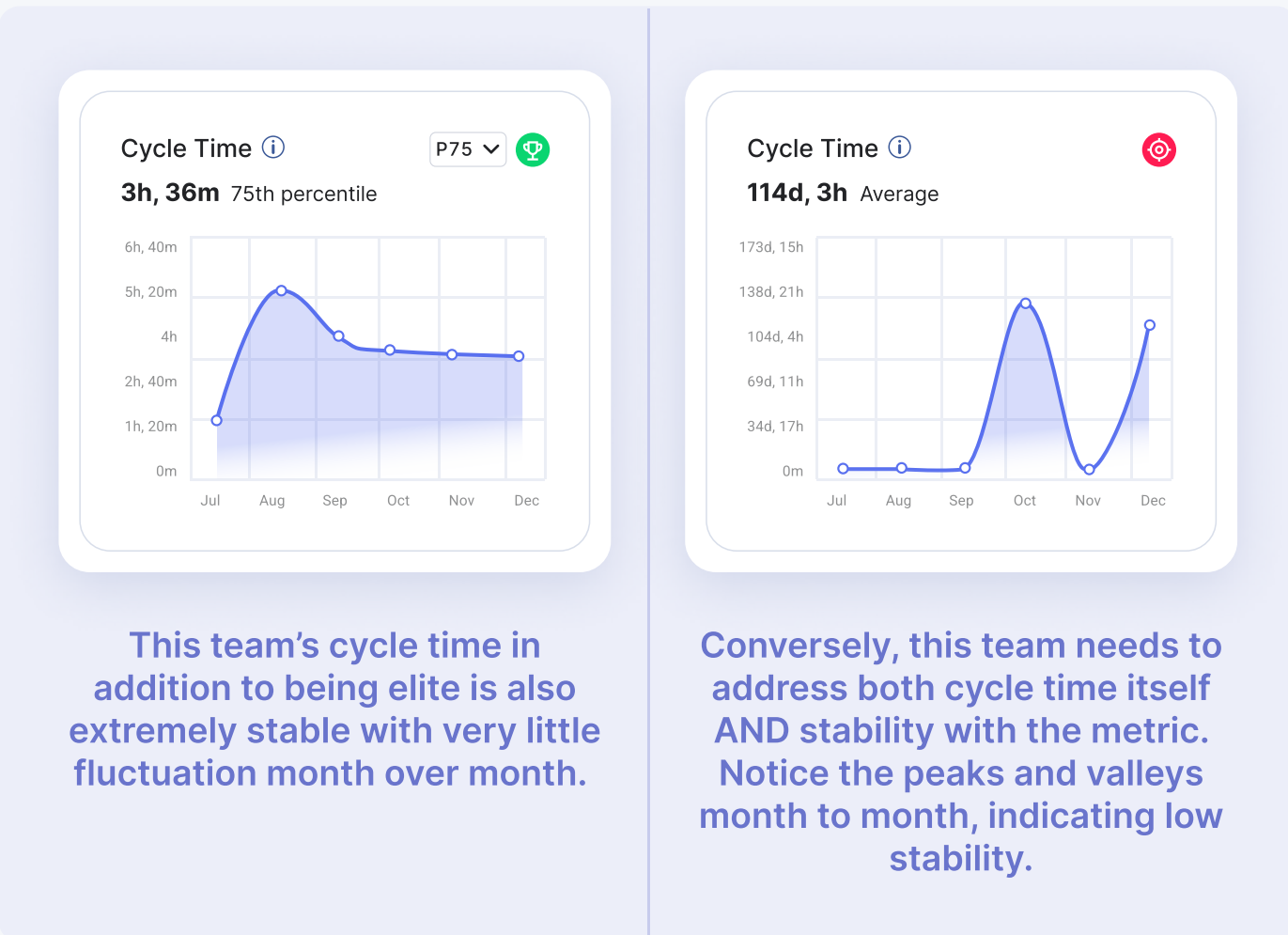
The Predictability Equation

We need to understand the predictability equation to avoid the pitfalls of unplanned work and stressed-out teams. It goes like this:
Stability + Velocity - Risk = predictability. We'll break down each variable in the equation in more detail in just a second, but always keep one thing in mind: every variable of the formula has a part to play in project predictability.



There's no denying that stability and velocity are essential factors for an efficient and predictable engineering organization, but they are not everything. You may be saying, "But wait, how can a team NOT be predictable if they're executing well?" For that answer, let's dive into each variable.

Stability



This team's cycle time (in addition to being elite) is highly stable, with tiny fluctuations month over month.

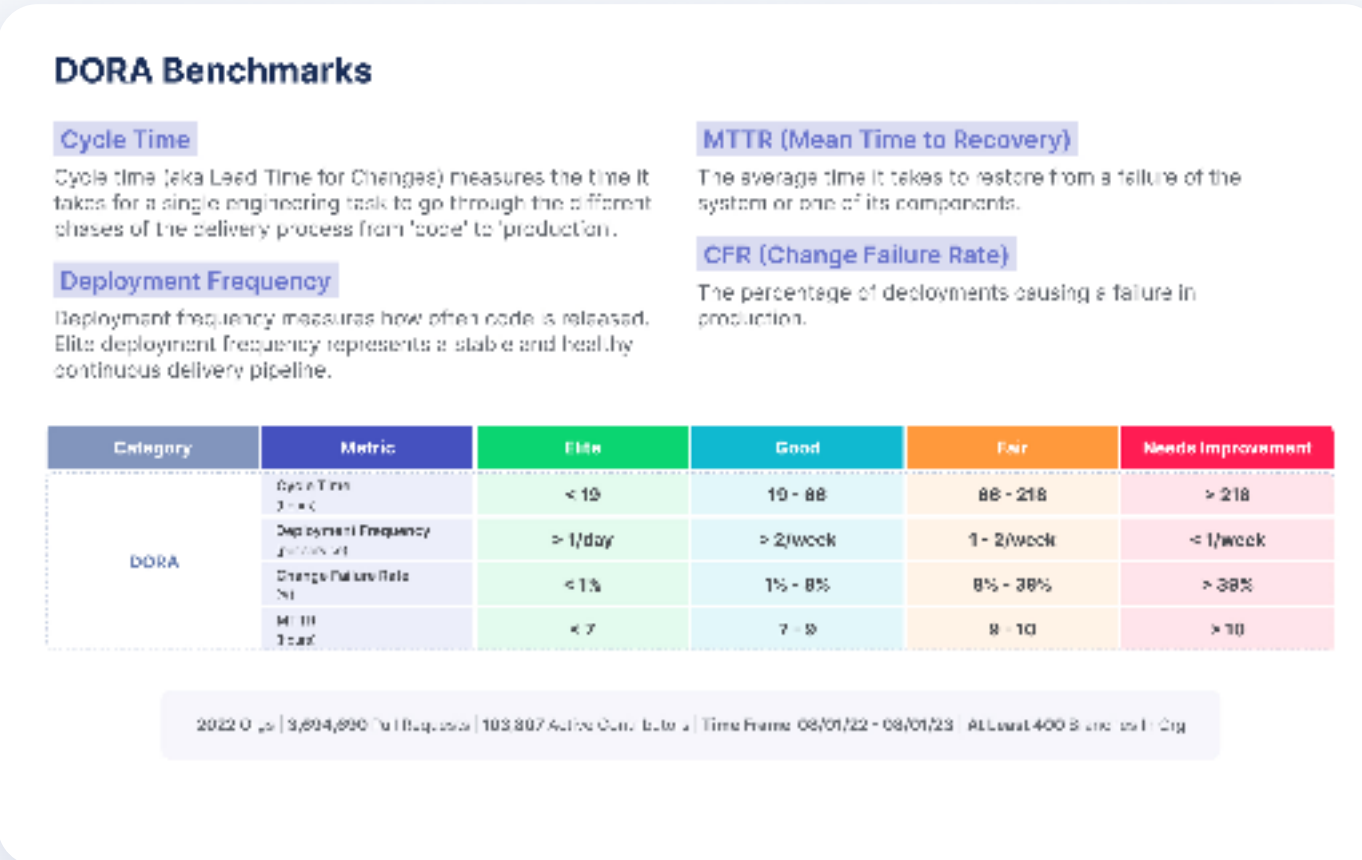
Conversely, this team needs to address both cycle time itself **AND** stability with the metric. Notice the peaks and valleys month to month, indicating low stability.

Here are some of the leading indicators that a team is pretty stable and consistent:

- Their cycle time remains consistent, iteration over iteration
- PR size is well in hand (<200 diffs)
- PRs are getting reviewed
- Rework rate is well within normal bounds (so they're not churning code)
- WIP for a given dev or team doesn't fluctuate greatly

Last, the team's **DORA metrics** are solid (at least "Good" in CFR and MTTR) as informed by **industry-standard benchmarks**. That means things aren't breaking in production at an alarming rate. However, when outages happen, they can respond quickly—a crucial consideration given that **just an hour of downtime costs SMBs nearly over \$25k while enterprises are out almost \$500k**.

Generally speaking, this team is trending toward being predictable.



The Predictability Equation

Velocity

That same team is also pretty healthy in their execution velocity (**at least "Good" in Cycle Time and Deployment Frequency**). Work is moving through the pipe and getting into production quickly, and the various phases of their overall process aren't raising any alarms. On the surface, this team's metrics look good, and it's likely they can predictably deliver projects.

But what if R&D investment and business priorities aren't perfectly aligned? They may have got too many people working on a deprioritized project. They may be too focused on enhancements to existing features that customers have yet to adopt widely. What if a new feature the business needs to land new customers or expand further into existing accounts gets deprioritized suddenly?

Given what you now know:

- The team's work is reasonably stable.
- They are fast in shipping their work.
- Team focus and business priorities aren't wholly aligned.

Is this team predictable? That's a more nuanced question with a less straightforward answer because of one factor: **risk**.

Risk

Risk is the chief variable in the predictability equation. Solid execution and high quality only amount to a little if you cannot derisk your R&D organization by addressing the common pitfalls many teams face. Let's break down each one in more detail:

Risk 1

Is unplanned work bogging the team down?

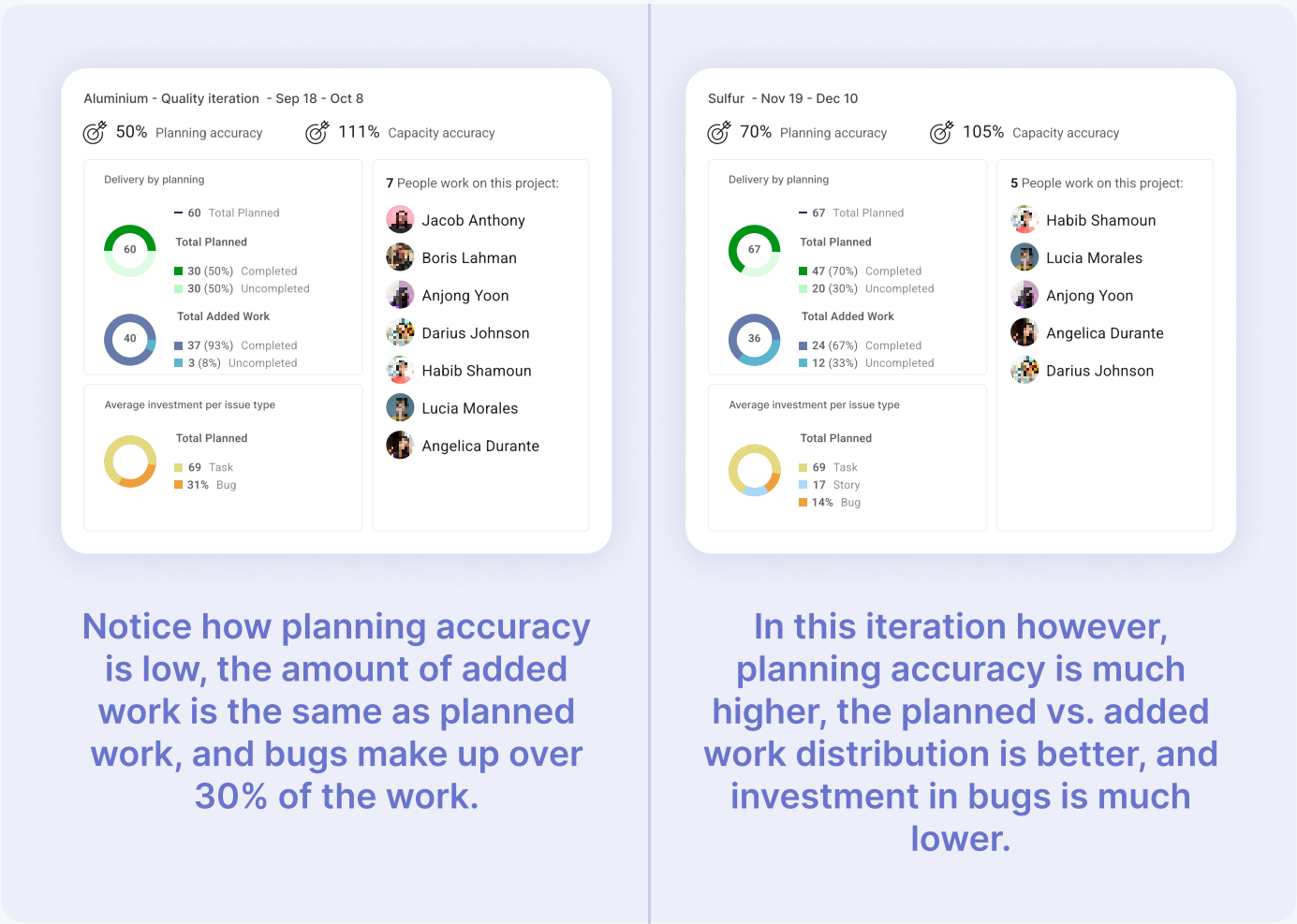
The ability to execute is a chief concern for every engineering leader and their teams. Maintaining focus needs to be as well. As R&D leaders know, story point estimation isn't exactly an exact science, and one team's 10 could be another team's 5. But assume that there's a standardized weighting for any given team.

Delivering 100 story points is an achievement, but not if 75 were bugs that got added mid-iteration. Following that trend, the likelihood of project/feature/product delays increases if most of the completed work has nothing to do with a promised feature or product enhancement. The key to avoiding that risk is ensuring teams prioritize scoped work.

Notice how planning accuracy is low, the amount of added work is the same as planned work, and bugs make up over 30% of the work.

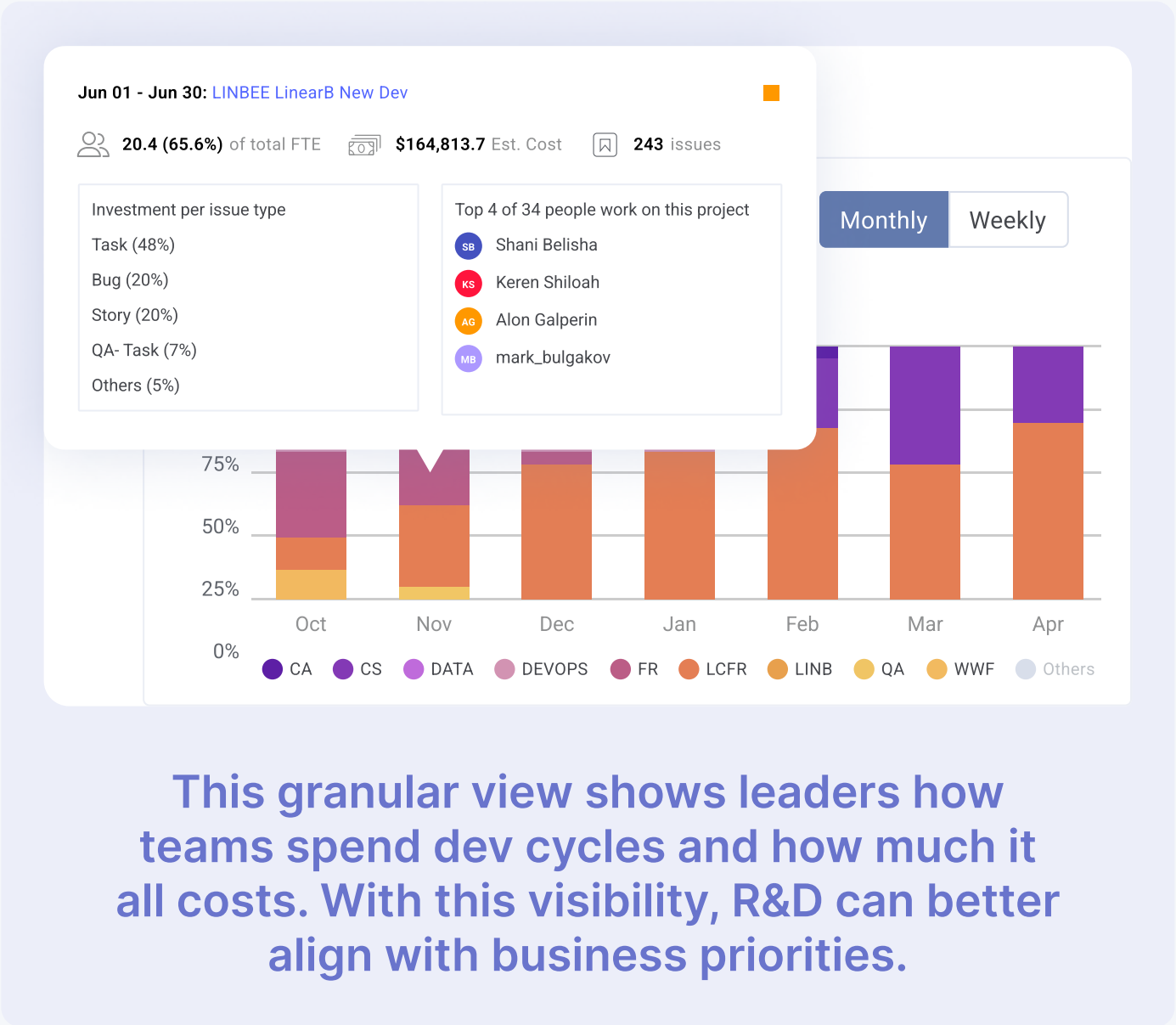
The Predictability Equation

However, in this iteration, planning accuracy is much higher, the planned vs. added work distribution is better, and investment in bugs is much lower.



The Predictability Equation

Bottom line: engineering budgets are finite and sometimes shrinking, so ensuring that the priority, high-impact projects have enough resources for on-time delivery is crucial to predictability. With visibility into project resourcing, engineering leaders can accomplish that goal and have meaningful discussions about adjusting strategies and optimizing R&D spend.



Risk 4

Are devs spending time on what will drive value for the organization?

Like resourcing, a non-optimized engineering investment strategy is an ever-present project delivery predictability risk.

Let's return to our example team with solid stability and velocity metrics. We know that this team can deliver quality work quickly.

But what happens to predictability if that team over-invests on things like Keeping the Lights On (KTLO) activities such as bug work, uptime maintenance, enhancements to or maintaining security posture, and general issue monitoring/triaging? That's right: time spent on New Value drivers suffers, including new features and roadmap items that increase revenue and growth by customer acquisition or expansion.

Without a significant investment in KTLO work, the business cannot function. But if a team spends too much time on that bucket, the risk of missing a promised deadline for delivering a new feature grows. This common scenario of unbalanced investments can often lead to negative CSAT scores, especially if the new feature in question is one that customers are counting on.

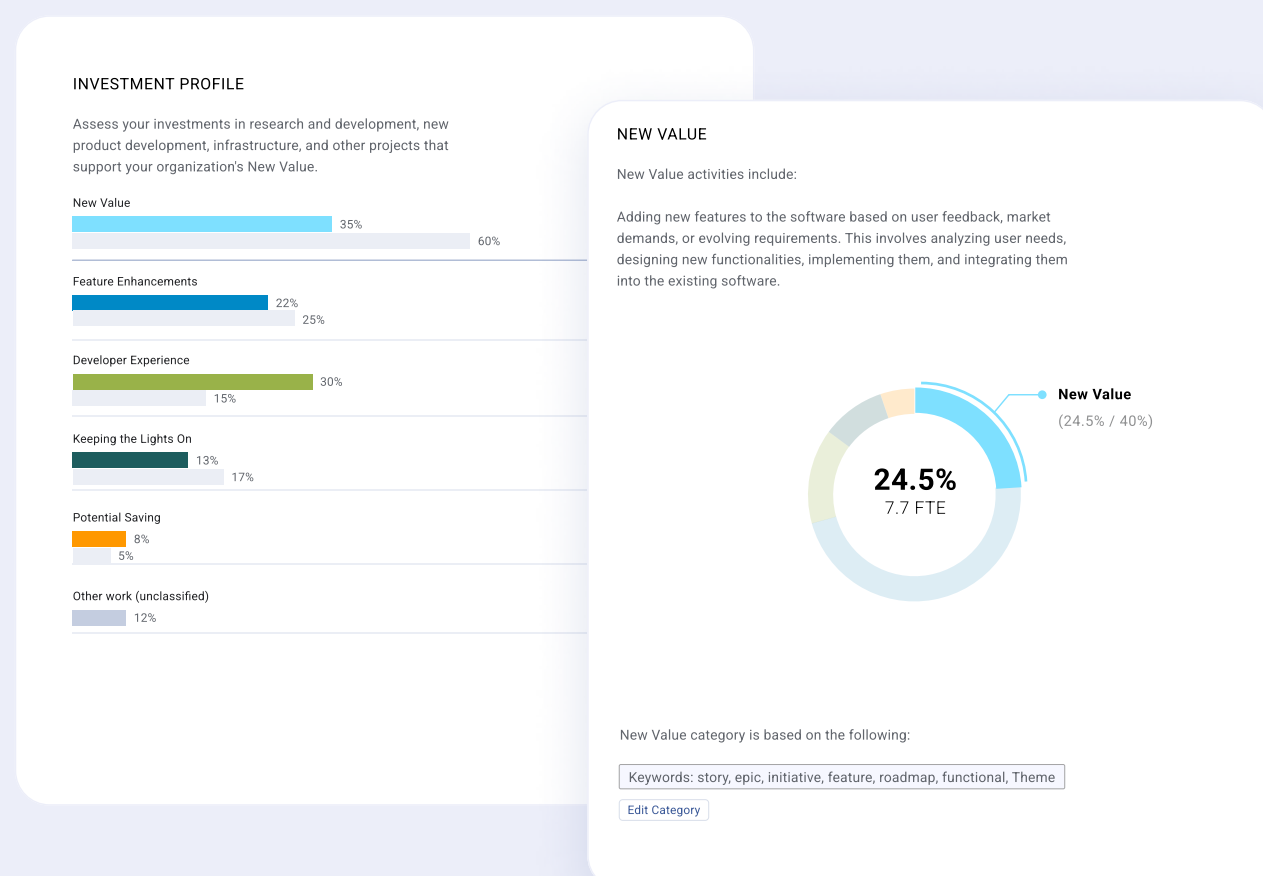
The Predictability Equation

Applying the right investment mix (**informed by industry benchmark data**) is essential for predictable project delivery and avoiding delay risk. Teams should also pay close attention to their "Inefficiency Pool," which is the potential value (dev hours) currently wasted due to process bottlenecks and idle time.

That means keeping things like WIP, velocity, stability, and added/unplanned work at acceptable levels every single iteration.

The best way to deliver consistency and ensure a great developer experience is through **targeted application of programmable workflow automation** that removes toil, reduces context switching, offloads manual work, minimizes human error, drives clarity and standardization, and gets developers back to what they love doing: coding.

The key to predictable project delivery is maintaining stability and velocity while minimizing risk.

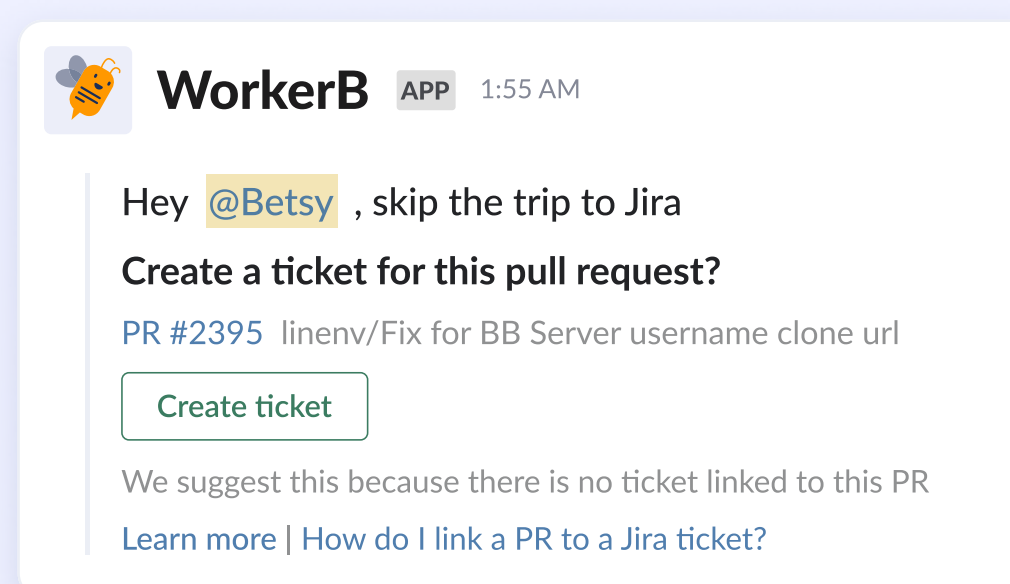


This team needs to adjust their investment profile and focus more time and effort on New Value creation.

Risk 5

Are developers happy, productive, and enjoying a good DevEx?

Developers want to be productive. Consistency and clarity in a working environment are conducive to productivity.



Automated alerts and one-click workflows remove toil, offload manual work, offset human error, and drive better habits.

How Syngenta Became More Predictable At Scale



Syngenta, a leader in agricultural technology, has what can be called an "existential" mission: helping millions of farmers worldwide grow food for billions of people. More than that, they help those farmers grow food efficiently, safely, and with a constant focus on sustainability and ecology.

By partnering with LinearB and aligning with the engineering success model, Syngenta was able to:

- Foster more ownership among their teams
- See a higher degree of planning success
- Gain more visibility into their work, which spurred better habits
- Adopted a transparent, sharing-focused culture that proactively communicates risk

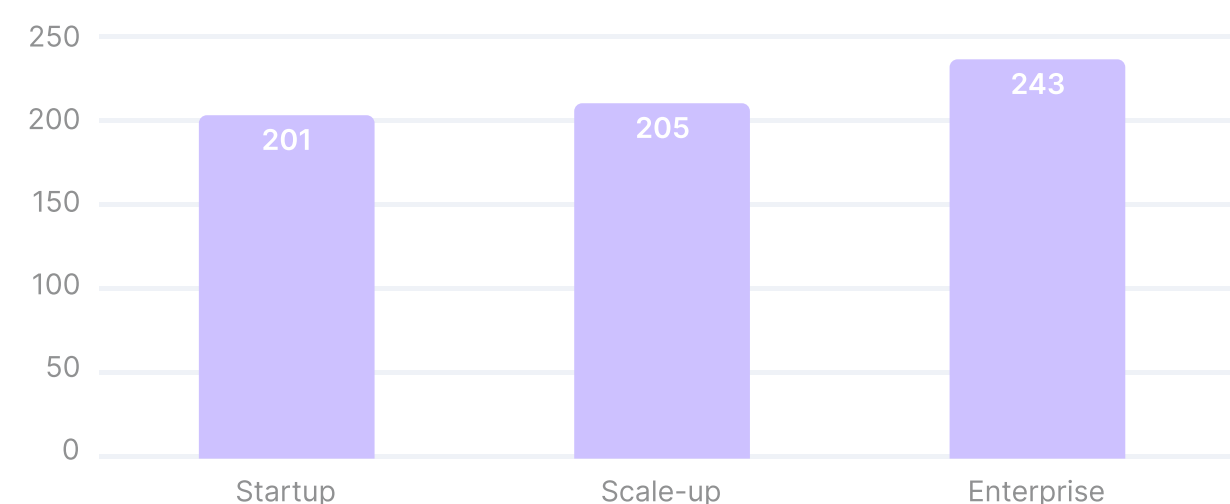
And they did it all while scaling their engineering organization from 150 to 400 developers.

The Challenge

The foundational problem Syngenta was trying to solve was scale; what had worked for a team of 150 no longer worked for a team of 400. In addition to scale, they faced other significant hurdles, including:

- A global presence that made collaboration challenging
- Distributed systems/teams (4 platforms and 11 consumer apps)
- Idle time and handoffs between teams when working asynchronously

Cycle Time By Org Size



Startup: 0-200 employees

Scale-up: 200-1000 employees

Enterprise: 1000+ employees

Data from the 2023 Software Engineering
Benchmarks Report

Key Takeaway

Enterprises have longer cycle times than startups and scale-ups.

They knew they needed visibility to answer three key questions:

- "Are we working efficiently?"
- "Are we working effectively?"
- "Are we working safely?"

In addition to answering those questions, Syngenta wanted to empower every level of R&D with more ownership over their metrics, the tools to identify their bottlenecks, and the autonomy to report their findings to leadership.

And all of this was in service to building a steady, predictable flow of feature delivery.

How Syngenta Became More Predictable At Scale

The Solution

Syngenta implemented a metrics program focused on core metrics that cut to the heart of engineering health. These KPIs included DORA metrics and leading indicators like PR size.



Cycle Time

Are we able to go from Idea to Production quickly?



Planning Accuracy and PR Size

Are we breaking down work enough to accurately move towards it?



Change Failure Rate

Are we able to move quickly without breaking things?

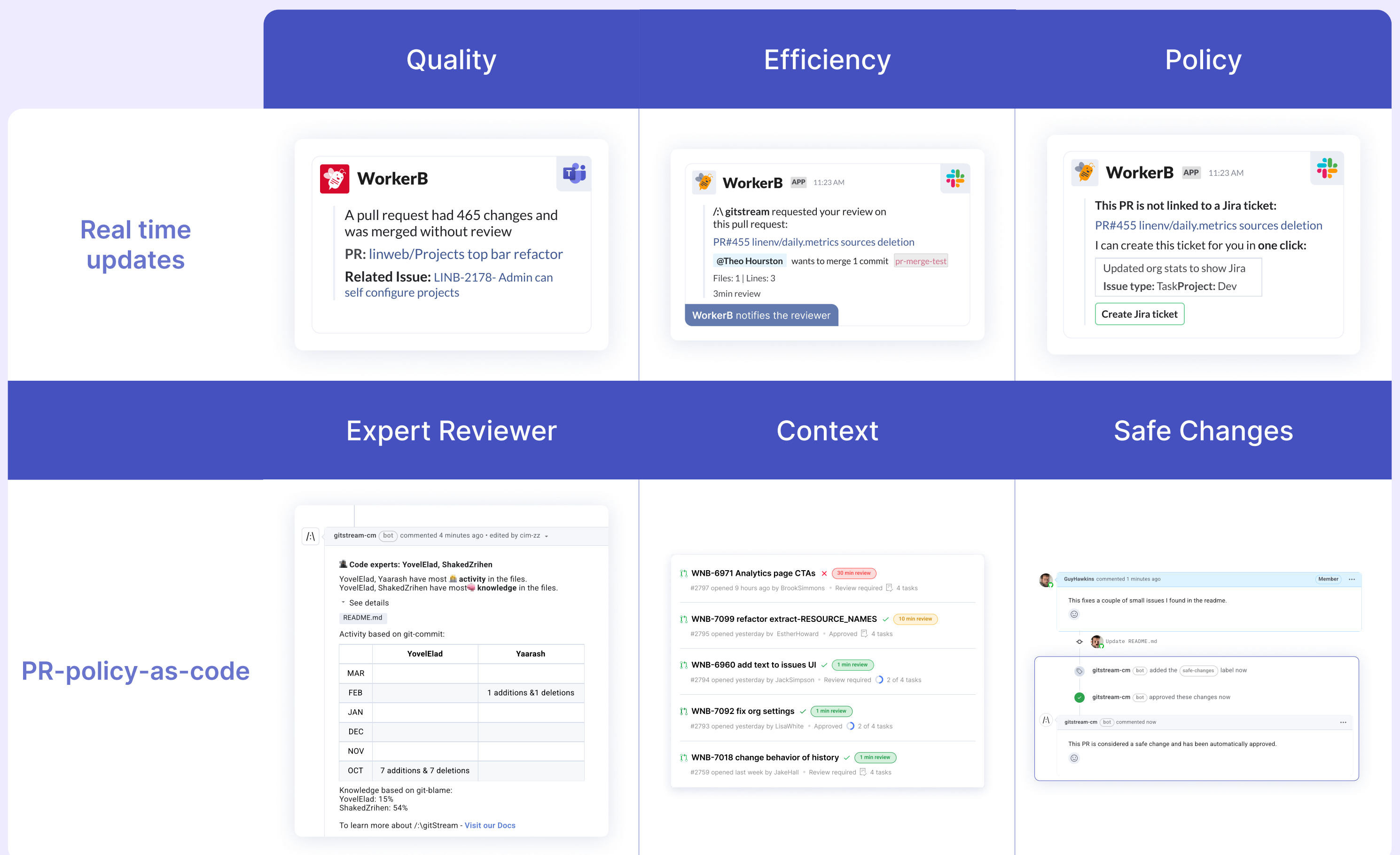


Mean Time to Recovery

If we do break them, can we detect and fix quickly?

How Syngenta Became More Predictable At Scale

In addition to tracking its metrics, Syngenta also focused on **doing something** with those metrics using programmable workflows. They adopted WorkerB and gitStream to remove software delivery friction, keep their developers aligned, provide a great DevEx, and enforce standards at scale.



How Syngenta Became More Predictable At Scale

The Results

Workflow automation became a critical tool across Syngenta's teams—driving significant reductions in PR pickup and review time. The increased visibility also exposed some missing best practices the team quickly addressed—specifically, more widespread adoption of a feature flagging platform to allow more frequent deployments.

Additionally, teams at Syngenta significantly matured functional capabilities with the ability to pinpoint inefficiencies in their operations and by transforming their ways of working—like putting more rigor around the definition of "done."

By identifying these bottlenecks and working to solve them with workflow automation, Syngenta reduced its **Cycle Time by 81% and increased Planning Accuracy by 33% in just six months!**

These remarkable improvements will help Syngenta become more predictable in project delivery as they continue their important work of supporting farmers around the world.

6 Month Results with LinearB



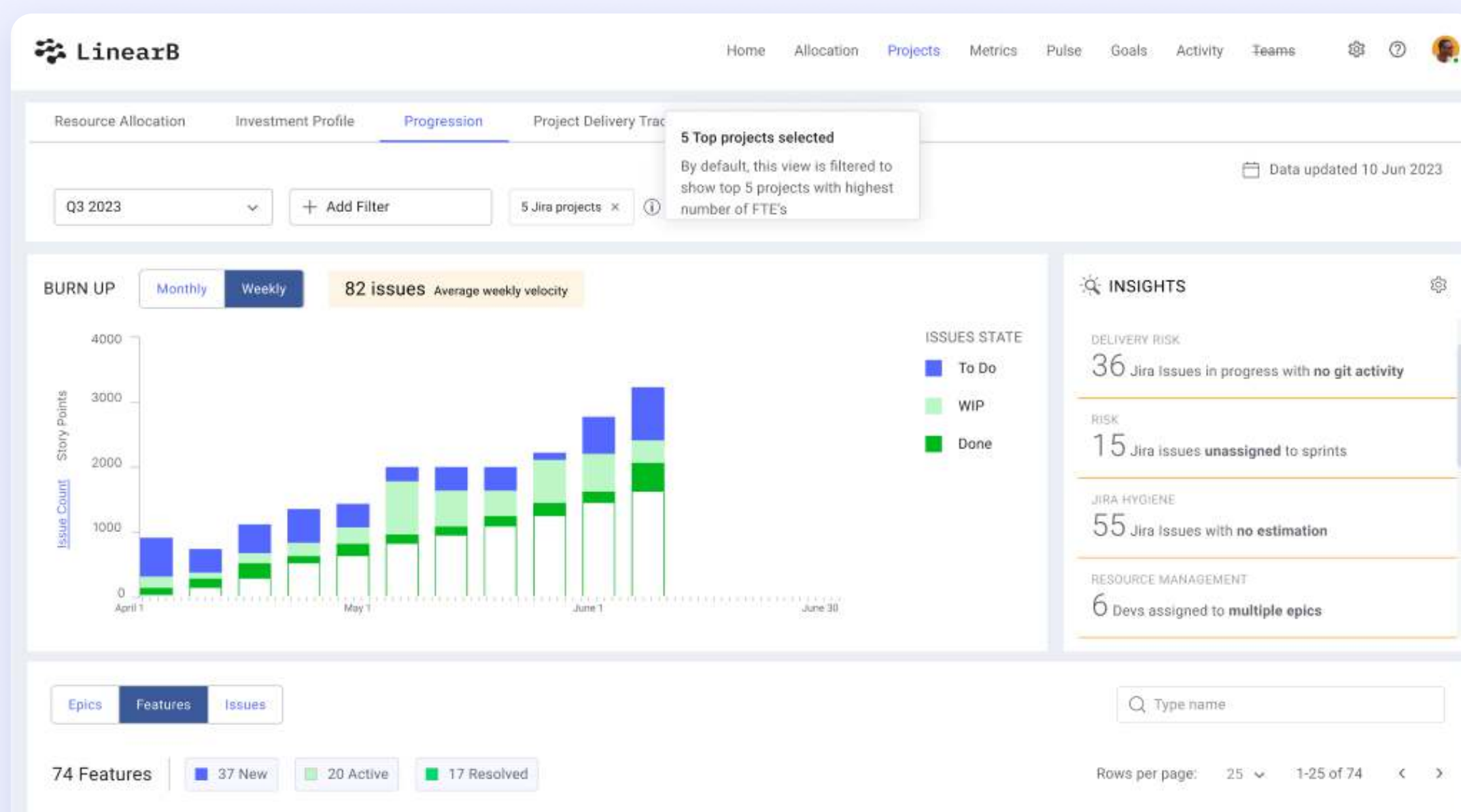
Cycle Time: 81% Decrease



Planning Accuracy: 33% Increase

Building a Predictable Project Delivery Engine

Project Progression is one of the best early indicators of whether your organization will deliver on time.



Predictability is the engineering promised land. When an engineering leader can confidently walk into the board room and present both project updates and delivery timelines that they KNOW are correct, that meeting goes from something to dread to an opportunity to show off.

The role of an engineering leader often involves acting as a translator between R&D and the rest of the business. When you set the right expectations (by providing accurate project delivery estimates, proactively communicating risk, and

maintaining focus on business imperatives), everyone in the room speaks the language better. With a baseline understanding, translation becomes more manageable. And **the difficult job of being an engineering leader** becomes exponentially easier.

Now that you understand the equation for project predictability, you need the right tools to solve it—just like mathematics requires a calculator.

Arriving at predictable project delivery comes down to:

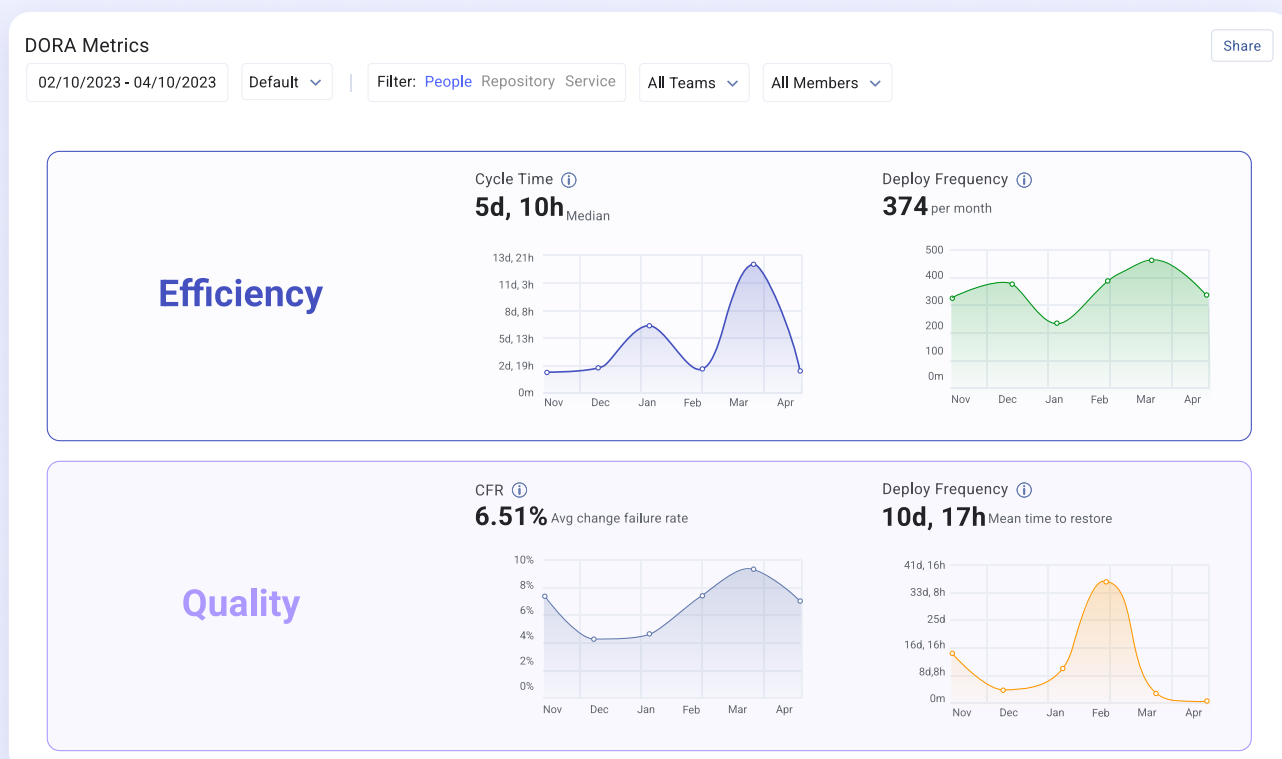
Building a Predictable Project Delivery Engine

Visibility, Benchmarks, and Diagnosis

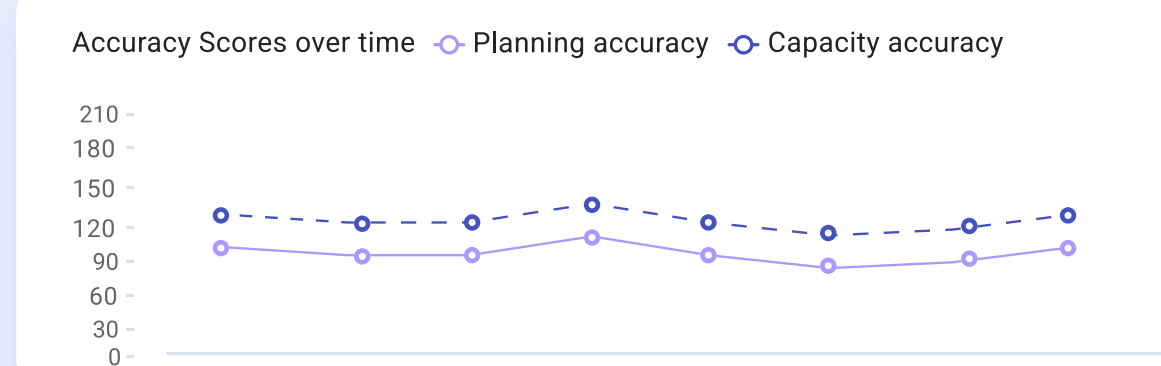
The first step toward consistent project delivery is data collection and analysis to glean necessary insight and identify gaps. And the easiest way to do that is by building an engineering metrics program. A metrics program is foundational to fulfilling the engineering leader's dual mandate of (1) delivering operational excellence and (2) driving better business results.

At a high level, the process of building an engineering metrics program comes down to:

- **Visibility:** See lagging and leading indicators of engineering health
- **Benchmarking:** Quantify and comprehend the unique definition of "good" for your team
- **Diagnosis and Reporting:** Decide which metrics matter to your team, impact the business, and need to be improved. Get on a regular cadence of highlighting successes and further opportunities.



DORA metrics are the foundational visibility you'll need for a metrics program and getting predictable with project delivery.



Be sure to incorporate Planning and Capacity Accuracy (as well as benchmarks) to see how well your teams are scoping and executing iterations—this is a great early indicator of project delivery.

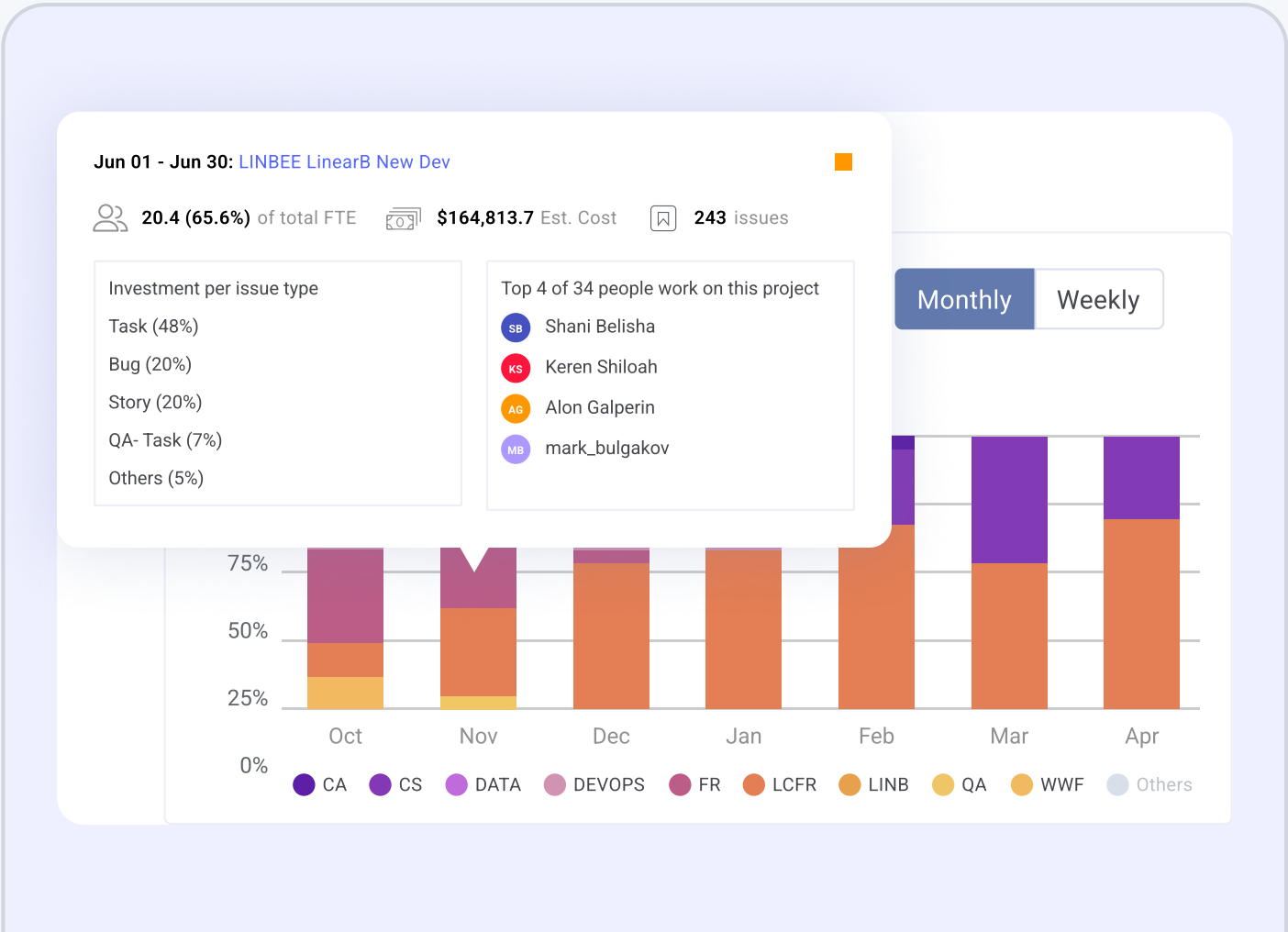
Building a Predictable Project Delivery Engine

Allocating Resources Correctly and Investing Wisely

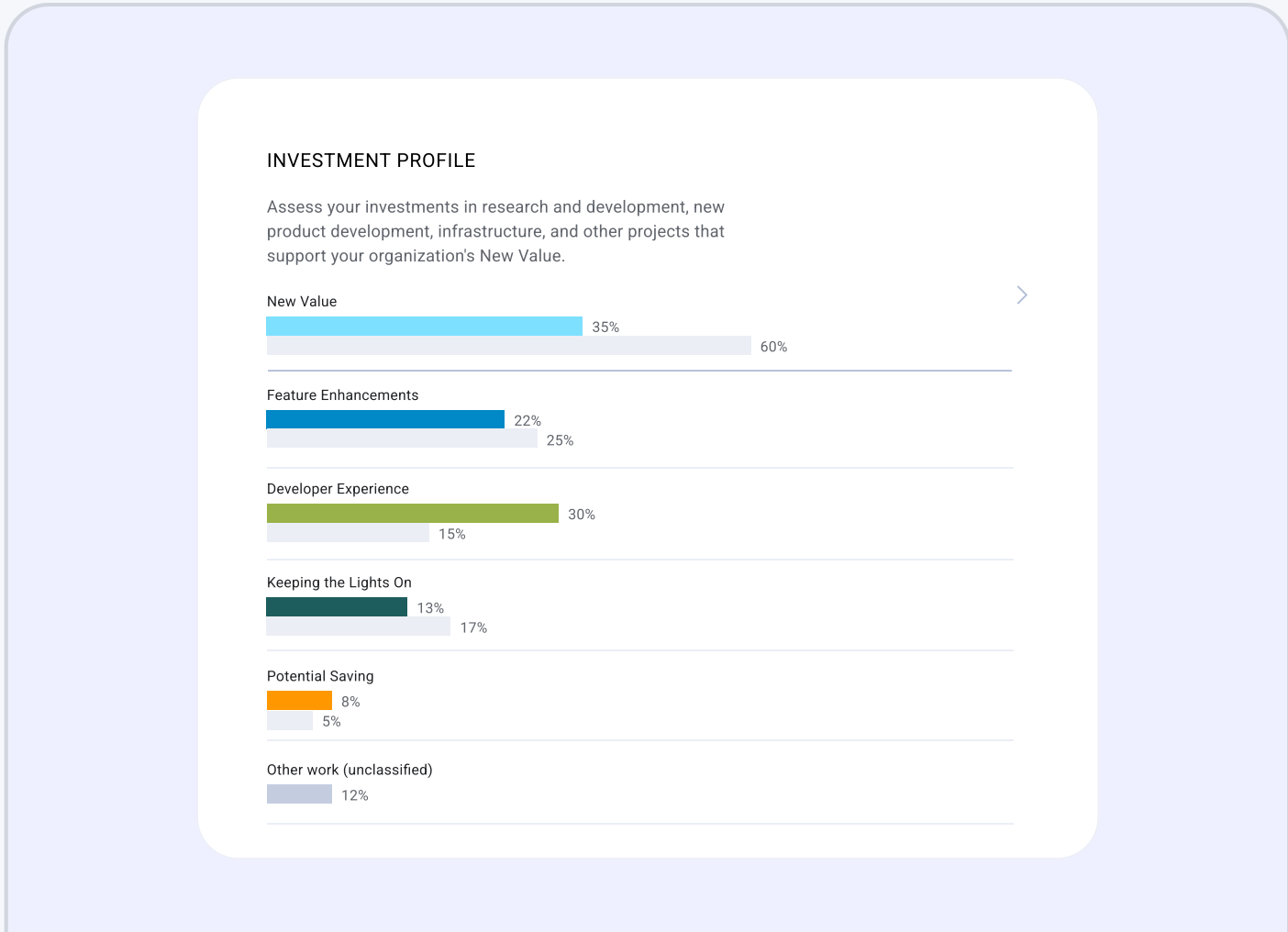
After you've got visibility into operational metrics, the next step in your predictability journey is to look at how well your engineering organization aligns with business priorities. That means analyzing and improving project resourcing and developer cycle investment. The ultimate goal of this step is to direct team focus and resources on getting what was promised out the door in the timeframe communicated to the business.

The standard operating procedure for getting engineering in lockstep with the business is essentially a three-step process:

- **Visibility:** Getting a holistic picture of what everyone in the organization is working on
- **Insights:** Using industry standards and historical patterns to find reallocation opportunities
- **Optimization:** Fine-tuning your project resourcing strategy to ensure engineering drives positive business outcomes



A breakdown of what every project costs and a great indicator of if you can deprioritize a project to move resources around to meet project goals.



Ensure organization-wide focus on what matters and what was promised (like New Value) by staying within investment benchmarks.

Building a Predictable Project Delivery Engine

Setting Data-Backed Improvement Goals

Once operational metrics telemetry, business alignment visibility, and a **working definition of "good" using industry benchmarks** are in place, identify your bottlenecks and set improvement goals against them.

Improving every aspect of your engineering organization will only sharpen your ability to deliver projects predictably—regardless of what kind of goals you're setting (operational, business, or ideally both).

Pro Tip(s)

- Set a maximum of 1 or 2 organizational improvement goals. Trying to improve everything all at once is a recipe for failure.
- Align goals at the org and team level. Start at the top and cascade goals down.
- Structure goals as OKRs and leverage the SMART framework.

Efficiency Metrics	
Cycle Time *	▲ 4d 1h
Deploy Frequency	9.2/ week
Merge Frequency	19.2/ week
PR Size	▲ 359
Coding Time	1h 36m
PR Pickup Time	▲ 2d 5h
PR Review Time	1d 19h
Deploy Time	23h

Quality Metrics	
Change Failure Rate*	7.47%
PRs Merged w/o Review	▲ 3.7/ week
Review Depth	1.03 comments/PR
PR Size	▲ 359
Rework Rate	▲ 17.7%
Non-functional work (%)	3.8%
Bug Investment	4%

Set goals based on identified bottlenecks

OBJECTIVE

Improve the efficiency of my engineering organization by End of Q1 (Objective)

Key Results

Reduce cycle time by 25%, 14% increase in deploy frequency

Further tactical key results: Keep PR size to <150 diffs, route PRs appropriately via automation (gitStream)

OBJECTIVE

Deliver on my promises to the business this quarter

Key Results

Maintain >75% Planning Accuracy, >85% Capacity Accuracy, and focus on planned work (fitting in unplanned/added work where possible)

Further tactical key results to get there: Keep PR size to <150 diffs, reduce cycle time by 20%, decrease CPR by 15%

OBJECTIVE

Allocate more of my engineering organization to high value/high impact projects (Objective)

Key Results

Maintain ~40% investment in New Value and ~30% investment in Feature Enhancements

We covered resource allocation in detail in the Engineering Leader's Guide to Resource Allocation

Some strategic operational and business goals structured as OKRs

WorkerB

A pull request has been merged containing 147 code changes with 82% refactor and new work

Goal: Avoid merging pull requests with work at risk

PR: Lindey Rovers "LINDEE-1002" Remove broken from Pulse CTS page

PR Owner: Chris Zelen

More logs PRs

WorkerB

A pull request took 23 days, 4 hours to be merged

Goal: Merge PRs in 4 days

PR: on-prem-agent/Linbee 443 create multi-pilot docker

More long lifespan PRs

WorkerB

Heads up: A pull request was opened with 967 code changes

Goal: Limit PRs to 300 code changes or less

PR: on-connectors/Azure report extend

PR Owner: Mark Budgekov

More logs PRs

A goals dashboard that you check regularly and automated alerting that aligns your team to stated improvement goals will help ensure success.

In addition to goal setting, establish a regular cadence of reporting on priority deliverables and strategies. Doing so will align both engineering and business stakeholders. Be sure to tailor your reporting to your audience. You can take much of the legwork out of this process by **plugging your data into this template**.

Building a Predictable Project Delivery Engine

Adopting Programmable Workflows

The final step is where the rubber meets the road. Operational metrics, business alignment visibility, and goal setting are only effective with a low-friction way to enact the changes engineering leaders want to see.

The key aspects to successfully leveraging programmable workflows are:

- Apply them in the environments where developers spend their time—namely communication tools and git repositories
- Use them to drive higher efficiency, increase quality, and **enforce standards at scale**
- Always measure process improvements so you can continue optimizing in the future



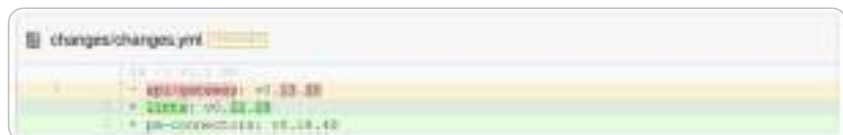
WorkerB APP 8:33 AM

Hey @Betsy , skip the trip to Jira

Miki requested your review for this pull request

[linenv/WWF-418-linrest-sensors-api-fix](#)

🕒 Est. review time: **up to 2 min** [Learn more](#)

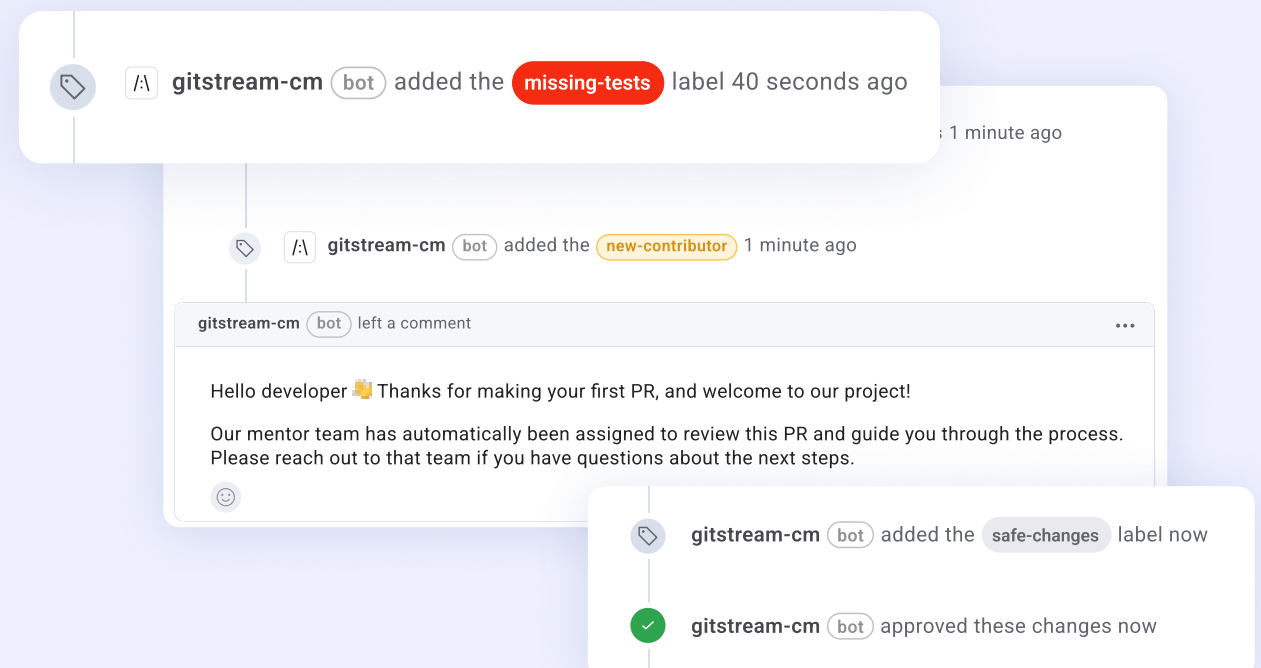


✅ Add your approval comment

LGTM

Approve

Automated alerting with WorkerB helps with goal alignment, reducing manual work, minimizing context switches, and ensuring work doesn't fall through the cracks.



Automated PR labeling with **gitStream** helps in many ways, including providing more context for reviewers, flagging issues, and assisting new developers in getting up to speed faster.

Building a Predictable Project Delivery Engine

Some checks haven't completed yet
1 action required and 5 successful checks

Hide all checks

/

-set-required-approvals@v1

Action required after 3s - Merge blocked - Wait for 2 more approvals

Details

/

-add-comment@v1

Successful in 4s - Comment added

Details

/

gitStream.cm

Successful in 1s - default/double_review, default/suggest_the_right_reviewer

RequiredDetails

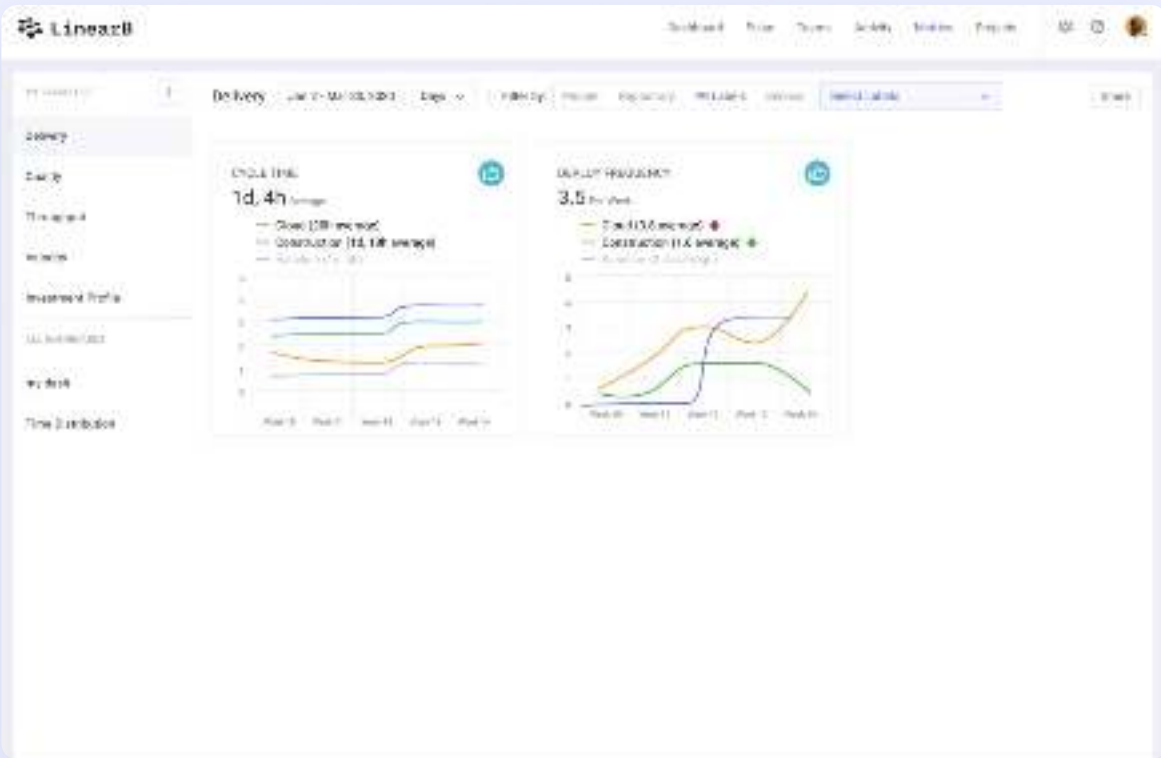
This branch has no conflicts with the base branch

Merging can be performed automatically

Merge pull request

You can also [open this in GitHub Desktop](#) or view [command line instructions](#).

gitStream can automatically trigger additional reviews to boost quality based on customizable criteria and standards for KPIs like PR size, PRs with security risks, PRs opened by new team members, or PRs that touch complex code.



Understand the impact of improvement initiatives and new processes—like adopting GenAI code tools—using automatically or manually assigned PR labels in concert with metrics dashboards.

Automation breeds efficiency and quality. A team that can maintain high levels of efficiency and quality is well on its way to predictable project delivery.

Conclusion



Congratulations, you're now well on your way to being able to predictably and repeatedly deliver projects on time, as promised. To recap, you have:

- Gained visibility via an engineering metrics program
- Ensured focus on business-impacting projects with Resource Allocation
- Fostered a culture of continuous improvement with goals and reporting
- Removed friction from the SDLC with automated programmable workflows

The result is the ability to:

1. Proactively identify risk indicators for project delays
2. Confidently communicate *accurate* project delivery forecasts consistently to stakeholders around the business.

Conclusion

LinearB's software engineering intelligence platform is ideal for building an engineering metrics program from scratch and aligning engineering to the needs of the business. After onboarding, initial configuration, and platform integration with your existing software stack, the metrics, dashboards, insights, and automation discussed in this guide are available out of the box.

You can **get your DORA metrics with a free forever account** and begin building your engineering metrics program today! To discuss the content in this guide in more detail or to see Predictable Project Delivery in action, you can **schedule a demo**.

We'd love to hear from you!